ECE 405, Spring 2026
CS336 Assignment 2 Basics
Filtering Language Modeling Data

Kanta Saito

Problem (look_at_cc): 4 points

(a) Download the WARC file above, or find the copy we provide on the cluster. Let's look at the first page in this file. This is a gzipped file, and you can browse its contents with:

```
$ zcat /data/CC/example.warc.gz | less
```

`less` lets you browse the file using keyboard arrows, Page Up, Page Down. To exit, press "q". Look at the very first web page. What is its URL? Is it still accessible? Can you tell what the page seems to be about by looking at the raw HTML?

Deliverable: A 2-3 sentence response.

Observations: The URL is `http://0371rykj.com/ipfhsb/34.html`. This link gives a "dangerous site" warning, so the page is currently inaccessible. Looking at the raw HTML, I noticed that the text contains Chinese characters. Additionally, the HTML contains the words "products" and "news," leading me to conclude that it might be a store website.

(b) Let's now look at the corresponding WET file:

```
$ zcat /data/CC/example.warc.wet.gz | less
```

Note that the WET files contain HTTP headers (e.g., `Content-Length`) that are not part of the extracted text contents. If you look at the first example, you will see that it contains text that was extracted from the raw HTML you just saw.

Notice that much of the extracted text is reminiscent of the HTML structure, and not actually the page's main content. Are there parts of the text you see that you think should have been filtered out by the extractor? Think about the quality of this text as training data: what might go wrong in training a model on text that looks like this? Conversely, what useful information can a model potentially extract from this page?

Deliverable: A 3-4 sentence response.

Observations: The text extraction process should have filtered out HTML boilerplate, such as navigation menus (e.g., "Home" or "Products"), as well as the explicit SEO spam keywords injected at the top of the page. Training a language model on this type of noisy data can cause it to learn disjointed syntactic structures or falsely associate legitimate scientific equipment with explicit adult content. Conversely, the only potentially useful information on this page is the actual text describing the company's instruments, which could provide valid Chinese industrial vocabulary.

(c) What makes a good training example is highly contextual. Describe an application domain for which this example might be useful to have in the training data, and one where it might not be.

Deliverable: A 1-2 sentence response.

Applications: This example would be valuable for spam-classification model. However, it would not be good when training a regular language model, which could lead to bad generation and poor grammatical fluency.

(d) Let's look at some more examples to get a better sense of what's in the Common Crawl. Look through 25 more WET records. For each record, very briefly comment on the document's language (if you can identify it), the domain name, what type of page it is, etc. How many examples does it take until you see what you'd deem a "high-quality" webpage?

Deliverable: Brief annotations of 25 documents with the document's language, domain, type of page, and any other miscellaneous notes about the document. The number of examples it takes until you see a high-quality example.

WET Record Annotations (25 Documents)

#	URL (domain)	Language	Type	Quality	Notes
1	0371rykj.com	Chinese (zho)	Product page (temp/humidity test chambers)	Low	Adult SEO spam injected at top
2	10www.chinatikfans.com	Chinese + English	Fan forum blog post (2010)	Low	Personal post about subtitling a Thai drama
3	13.usnccm.org	English	Academic conference homepage (2015)	**High**	US National Congress on Computational Mechanics — real substantive content
4	176.utchat888.com	Chinese	Adult webcam/live chat platform (Taiwan)	Low	Adult content
5	176766.cn	Japanese + Chinese	Industrial testing equipment catalog	Low	Adult SEO spam injected at top
6	178mh.com	Chinese	404 / broken template error page	None	No content
7	1796370.tgtg97.com	Chinese	Adult livestream — streamer profile (Taiwan)	Low	Adult content
8	18sex.v340.info	Chinese + English	Adult webcam rankings page (Taiwan)	Low	Adult content
9	1kb.klimtoren.be	Dutch + English	Teacher's classroom blog post	Low	Very short birthday announcement, minimal information
10	1pekesat-exae.mysch.gr	Greek + English	Greek school distance-ed helpdesk — forum search UI	Low	Mostly navigation chrome, no content
11	1pekesat-exae.mysch.gr (login)	Greek + English	Login/authentication page	None	No substantive content
12	1s6605084.yhxzseo.com	Chinese	SEO spam article (lottery/betting app promo)	Low	Spam content injected into a health site template
13	20com20.fr/manual/tr/sitemap.html	Turkish + Danish + English	Apache HTTP Server 2.4 docs sitemap (Turkish)	Medium	Legitimate technical documentation mirror
14	24ktcasino.net	English	Casino affiliate blog post (Laos casinos)	Low	Thin affiliate/SEO content
15	2kgames.eu	English	404 Not Found (nginx)	None	No content
16	2l6185919.yizhangting.com	Chinese	SEO spam article (lottery app promo)	Low	Spam injected into health portal template
17	303323.com	Chinese	Medical device company news article	Medium	Substantive article on electrosurgical cutters for minimally invasive GI surgery
18	30bad.com	Chinese	Piracy streaming site — anime listing page	Low	No original content
19	312001.net	Chinese	Community health center CMS navigation/listing	Low	Mostly nav links, very little prose
20	354577.mwe075.com	Chinese	Adult webcam — streamer profile (Taiwan)	Low	Adult content
21	356.schoollibrary.edu.pe.ca	English	PEI School Library (Canada) — Koha catalog search results	Medium	Legitimate library catalog; structured but real data
22	366392.haaxz.com	Chinese	Adult webcam — streamer profile (Taiwan)	Low	Adult content
23	366392.haaxz.com (2nd)	Chinese	Adult webcam — streamer profile (Taiwan)	Low	Adult content, duplicate domain
24	387tel.com	Chinese + English	Adult video chat — streamer profile (Taiwan)	Low	Adult content
25	3diasdemarzo.blogspot.com	Spanish	Political/journalistic blog post (2005, Spain)	**High**	Substantive reporting on the 11-M investigation; real journalism

Figure 1: WET Record Annotations

A first high-quality example: Record 3 (13.usnccm.org)

The vast majority of the records are of low quality, primarily consisting of adult content sites (approximately 10 out of 25) and empty or error pages (around 3 out of 25). Only 2 of the 25 records (Records 3 and 25) could be considered high-quality, with 3 others (Records 13, 17, and 21) classified as medium-quality. This demonstrates that high-quality content is rare in this dataset and requires significant filtering to surface.

Problem (extract_text): 3 points

(a) Write a function that extracts text from a byte string containing raw HTML. Use `resiliparse.extract.html2text` to perform the extraction. This function needs a string, so you will need to first decode the byte string into a Unicode string. Be aware that the input byte string might not be encoded in UTF-8, so your function should be able to detect the encoding in case UTF-8 fails. Resiliparse also offers `resiliparse.parse.encoding.detect_encoding()`, which might be useful.

Deliverable: A function that takes a byte string containing HTML and returns a string containing the extracted text. Implement the adapter `[run_extract_text_from_html_bytes]` and make sure it passes `uv run pytest -k test_extract_text_from_html_bytes`

Implementation: I implemented a `try/except` block to catch any `UnicodeDecodeErrors`. If an error is caught—indicating that the byte string is not encoded in UTF-8—the function detects the correct encoding type and successfully decodes the HTML accordingly.

Code: Code is in `cs336-data/cs336_data/implementation.py`.

(b) Run your text extraction function on a single WARC file. Compare its output to the extracted text in the corresponding WET file. What differences and/or similarities do you notice? Which extraction seems better?

Deliverable: 2-3 sentence response comparing and contrasting the text extracted by your own function versus the extracted text in the WET files.

Response: My custom extraction function successfully pulled the text content from the WARC file, but it failed to strip out structural HTML tags like `<ul>` and `<th>`, leaving messy markup in the output. In contrast, the corresponding WET file contains clean, fully stripped plaintext. Therefore, the WET file's extraction is significantly better because it successfully isolates the human-readable text without the noise of leftover HTML structure.

Code: Code is in `cs336-data/cs336_data/tests.ipynb`

Problem (language_identification): 6 points

(a) Write a function that will take a Unicode string and identify the main language that is present in this string. Your function should return a pair, containing an identifier of the language and a score between 0 and 1 representing its confidence in that prediction.

Deliverable: A function that performs language identification, giving its top language prediction and a score. Implement the adapter `[run_identify_language]` and make sure it passes both tests in `uv run pytest -k test_identify_language`. Note that these tests assume a particular string identifier for English ("en") and Chinese ("zh"), so your test adapter should perform any applicable re-mapping, if necessary.

Implementation: I implemented the language identification function using the provided FastText model. The function passes the input text to the model to predict the most likely language label and extracts its corresponding confidence probability.

Code: Code is in `cs336-data/cs336_data/implementation.py`.

(b) The behavior of language models at inference time largely depends on the data they were trained on. As a result, issues in the data filtering pipeline can result in problems downstream. What issues do you think could arise from problems in the language identification procedure? In a higher-stakes scenario (such as when deploying a user-facing product), how would you go about mitigating these issues?

Deliverable: A 2-5 sentence response.

Response: Problems in the language identification procedure can lead to the disproportionate filtering of non-English text, resulting in severe vocabulary gaps and bias in the training data. At inference time, these deficiencies can cause the model to generate culturally inaccurate responses, struggle with multilingual contexts, or completely exclude non-English-speaking users. To mitigate these issues in a high-stakes scenario, I would deploy an ensemble of language classifiers specifically tuned to handle diverse character sets and mixed-language documents.

(c) Run your language identification system on text extracted from the WARC files (via your previously-implemented text extraction function). Manually identify the language in 20 random examples and

compare your labels with the classifier predictions. Report any classifier errors. What fraction of documents are English? Based on your observations, what would be a suitable classifier confidence threshold to use in filtering?

Deliverable: A 2-5 sentence response.

Response: Out of the 20 randomly sampled documents, exactly 6 were English. Surprisingly, there was only one misclassification, as shown in Figure 2. Based on these observations, a suitable classifier confidence threshold to use for filtering would be 0.60.

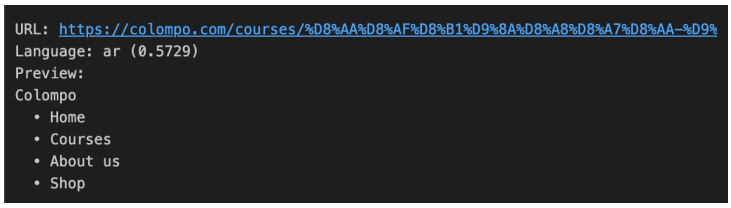

Figure 2: Classifier Error

Code: Code is in `cs336-data/cs336_data/tests.ipynb`

Problem (mask_pii): 3 points

1. Write a function to mask out emails. Your function will take a string as input, and replace all instances of email addresses with the string `"|||EMAIL_ADDRESS|||"`. To detect email addresses, you can look up regular expressions that do this reliably.

   Deliverable: A function that replaces all email addresses in a given string with the string `"|||EMAIL_ADDRESS|||"`, returning a pair containing both the new string and the number of instances that were masked. Implement the adapter `[run_mask_emails]` and make sure it passes all tests in `uv run pytest -k test_mask_emails`.

   Implementation: I implemented the email masking function using the following regular expression pattern:
   `r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'`. Using this pattern, I replaced all matching email addresses with the `"|||EMAIL_ADDRESS|||"` token and determined the total number of replacements by evaluating the length of the `re.findall` output.

   Code: Code is in `cs336-data/cs336_data/implementation.py`.

2. Write a function to mask out phone numbers. Your function will take a string as input, and replace all instances of phone numbers with the string `"|||PHONE_NUMBER|||"`. Doing this reliably can be extremely challenging, as phone numbers might be written in an extremely diverse set of formats, but you should try to capture at least the most common phone number formats used in the United States, and be robust to minor syntactic deviations.

   Deliverable: A function that replaces phone numbers in a given string with the string `"|||PHONE_NUMBER|||"`, returning a pair containing both the new string and the number of instances that were masked. Implement the adapter `[run_mask_phone_numbers]` and make sure it passes `uv run pytest -k test_mask_phones`.

   Implementation: I implemented the phone number masking function using the following regular expression pattern:
   `r'(\+?\d{1,3}[-.\s]?)?(\(?\d{3}\)?[-.\s]?)?\d{3}[-.\s]?\d{4}'`. Using this pattern, I replaced all matching phone numbers with the `"|||PHONE_NUMBER|||"` token and counted the instances using the length of the `re.findall` output.

   Code: Code is in `cs336-data/cs336_data/implementation.py`.

3. Write a function to mask out IP addresses. For this problem, it is enough to focus on IPv4 addresses (4 numbers up to 255 separated by points). Your function will take a string as input, and replace all instances of IP addresses with the string `"|||IP_ADDRESS|||"`.

   Deliverable: A function that replaces IPv4 addresses in a given string with the string `"|||IP_ADDRESS|||"`, returning a pair containing both the new string and the number of instances that were masked. Implement the adapter `[run_mask_ips]` and make sure it passes `uv run pytest -k test_mask_ips`.

   Implementation: I implemented the IPv4 address masking function using the following regular expression pattern:
   `r'\b(?:\d{1,3}\.){3}\d{1,3}\b'`. Using this pattern, I replaced all matching IP addresses with the `"|||IP_ADDRESS|||"` token and counted the instances using the length of the `re.findall` output.

   Code: Code is in `cs336-data/cs336_data/implementation.py`.

4. What problems do you think might arise downstream in a language model when these filters are naïvely applied on the training set? How might you mitigate these issues?

   Deliverable: A 2-5 sentence response.

   Response: Naïvely applying these PII filters to the training set can disrupt the syntactic structure of the text and accidentally erase valuable, historically significant public information. Consequently, the downstream language model might struggle to generalize to real-world data or become confused by the sudden appearance of artificial tokens. A robust mitigation strategy is to use format-preserving masking, which replaces the detected PII with synthetically generated, realistic counterparts (e.g., swapping a real IP address with a fake, randomly generated IP address) to maintain the natural flow and formatting of the text.

5. Run your PII masking functions on text extracted from the WARC files (via your previously-implemented text extraction function). Look through 20 random examples where a replacement was made; give some examples of false positives and false negatives.

   Deliverable: A 2-5 sentence response.

(a) False Positive Example 1 (b) False Positive Example 2 (c) False Negative Example

Figure 3: False Positives and False Negatives from random examples

   Response: Upon reviewing 20 random examples of PII replacements, the regex functions successfully caught many standard formats but also produced clear errors. Common false positives occurred when the phone number filter aggressively masked digit sequences embedded within URLs or copyright strings, such as converting a URL parameter to `https://alones.kr/?p=|||PHONE_NUMBER|||`. Conversely, a notable false negative occurred when a legitimate international phone number formatted with spaces (`+38 095 053 08 48`) was entirely missed by the filter, even though an email address in the exact same document was successfully masked.

   Code: Code is in `cs336-data/cs336_data/tests.ipynb`

Problem (harmful_content): 6 points

1. Write a function to detect NSFW content.

   Deliverable: A function that labels a given string as containing NSFW content or not, returning a pair containing both the label and a confidence score. Implement the adapter `[run_classify_nsfw]`

and make sure it passes `uv run pytest -k test_classify_nsfw`. Note that this test is just a sanity check, taken from the Jigsaw dataset, but by no means asserts that your classifier is accurate, which you should validate.

Implementation: I implemented the NSFW content classifier using the `fasttext` library. To avoid reloading the model on every call, I load the pre-trained `jigsaw_fasttext_bigrams_nsfw_final.bin` model globally. I preprocess the input string by replacing newline characters with spaces, pass it to the model's `predict` method, and then strip the `__label__` prefix from the output to return the clean label and its corresponding confidence score.

Code: Code is in `cs336-data/cs336_data/implementation.py`.

2. Write a function to detect toxic speech.

   Deliverable: A function that labels a given string as consisting of toxic speech or not, returning a pair containing both the label and a confidence score. Implement the adapter `[run_classify_toxic_speech]` and make sure it passes `uv run pytest -k test_classify_toxic_speech`. Again, this test is just a sanity check, also taken from Jigsaw.

   Implementation: I implemented the toxic speech classifier using a similar approach with the `fasttext` library. I load the `jigsaw_fasttext_bigrams_hatespeech_final.bin` model, sanitize the input text by replacing any newline characters with spaces to ensure accurate evaluation, and generate a prediction. Finally, I remove the `__label__` prefix from the predicted class and return the toxic speech label alongside its probability score.

   Code: Code is in `cs336-data/cs336_data/implementation.py`.

3. What problems do you think might arise downstream in a language model when these filters are applied to create the training set? How might you mitigate these issues?

   Deliverable: A 2-5 sentence response.

   Response: Applying strict toxic speech and NSFW filters to a training set often results in the erasure of marginalized voices and the unjust filtering of medical or educational content, leading to a downstream language model that is overly sanitized and suffers from excessive refusal. To mitigate these issues, rather than dropping the flagged text entirely, the training data can be annotated with control tokens. This allows the model to explicitly learn the distribution of harmful content, making it better equipped to avoid generating such content during inference time without losing valuable contextual knowledge.

4. Run your harmful content filters on text extracted from the WARC files (via your previously-implemented text extraction function). Look through 20 random examples and compare the classifier predictions to your own judgments. Report any classifier errors. What fraction of documents are harmful? Based on your observations, what would be suitable classifier confidence threshold(s) to use in filtering?

   Deliverable: A 2-5 sentence response.

   Response: After manually reviewing 20 randomly sampled documents, none were genuinely harmful, yielding a harmful fraction of 0/20. The classifier performed well on these samples, correctly identifying safe content—such as golf associations and standard web menus—with high confidence scores (e.g., 0.99 to 1.0 for `non-nsfw` and `non-toxic`). Based on these observations, a relatively high confidence threshold of 0.85 or 0.90 for the harmful labels would be suitable to prevent the aggressive over-filtering of benign content.

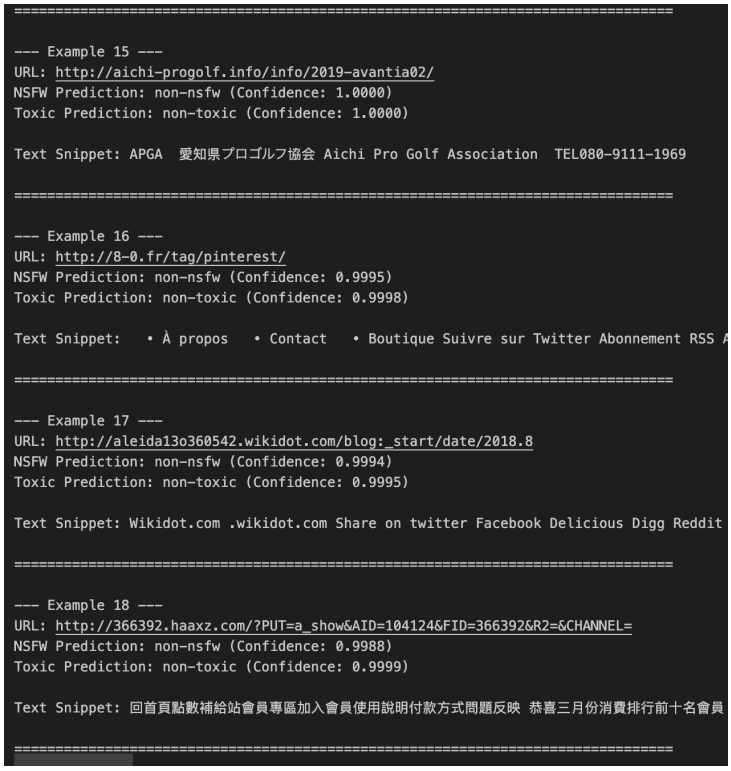

Figure 4: Harmful Content Review Examples

Code: Code is in `cs336-data/cs336_data/tests.ipynb`

Problem (gopher_quality_filters): 3 points

(a) Implement (at least) the subset of the Gopher quality filters as described above. For tokenizing text into words, you might find the NLTK package useful (specifically `nltk.word_tokenize`), though you're not required to use it.

Deliverable: A function that takes a string as its only argument and returns a boolean indicating whether the text passes the Gopher quality filters. Implement the adapter `[run_gopher_quality_filter]`. Then, make sure your filters pass the tests in `uv run pytest -k test_gopher`.

Implementation: I implemented the Gopher quality filters by first tokenizing the input text into words using `nltk.word_tokenize`. The function then evaluates the text against four specific rules: the document must contain between 50 and 100,000 words, possess a mean word length between 3 and 10 characters, have fewer than 30% of its lines ending in an ellipsis, and ensure at least 80% of the words contain at least one alphabetic character. If the text fails any of these checks, the function returns `False`; otherwise, it returns `True`.

Code: Code is in `cs336-data/cs336_data/implementation.py`.

(b) Run your rule-based quality filter on text extracted from the WARC files (via your previously-implemented text extraction function). Look through 20 random examples and compare the filter predictions to your own judgment. Comment on any cases where the quality filters differ from your judgments.

Deliverable: A 2-5 sentence response.

Response: After manually reviewing 20 random examples, I determined that only 2 of the 20 documents were genuinely high-quality text suitable for language model training. A significant discrepancy between the filter's predictions and my own judgment occurred with non-English data; for example, the filter incorrectly labeled a fragmented Chinese navigation menu as `PASSED (High Quality)`. This highlights a major flaw in the pipeline: because Chinese text lacks whitespace, the

nltk tokenizer fails to split words accurately. This demonstrates that these rigid, English-centric heuristics are highly unreliable for multilingual datasets.

Code: Code is in cs336-data/cs336_data/tests.ipynb

Problem (quality_classifier): 15 points

(a) Train a quality classifier that, given text, returns a numeric quality score.

Deliverable: A quality classifier for use in the next subproblem.

Implementation: To train the quality classifier, I first constructed a balanced dataset by processing two WARC files. For the negative examples (__label__lq), I extracted 5,000 documents from the Common Crawl dataset, filtering out non-English text using my language_identification function. For the positive examples (__label__hq), I extracted 5,000 documents from the subsampled_positive_ur file, applying both the language filter and my gopher_quality_filter to ensure high text quality. After combining and shuffling these datasets into train_shuffled.txt, I used fasttext.train_supervised to train the model with hyperparameters set to 25 epochs, a learning rate of 0.1, and word n-grams of 2. The trained model was then saved as quality_classifier.bin, achieving a precision and recall of 0.910 on the training data.

(b) Write a function that labels a page as high or low-quality, and provides a confidence score in the label.

Deliverable: A function taking a string as its only argument, and returning a pair with a label (high-quality or not) and a confidence score. Implement the adapter [run_classify_quality]. As a sanity check, make sure it correctly classifies the two examples we provide by running uv run pytest -k test_classify_quality.

Implementation: I implemented the quality_classifier function by loading the custom quality_classifier.b model that I trained in the previous step. The function takes the raw input text, sanitizes it by stripping newline and carriage return characters, and then passes it to the FastText model's predict function. It removes the __label__ prefix from the output and returns the classification result ("hq" or "lq") alongside the float representation of the confidence score.

Code: Code is in cs336-data/cs336_data/implementation.py.

Problem (exact_deduplication): 3 points

Write a function that takes a list of paths to input files and performs exact line deduplication on them. It should first count the frequency of each line in the corpus, using a hash to reduce memory, and then rewrite each file by only keeping its unique lines.

Deliverable: A function that performs exact line deduplication. Your function should take two arguments: (a) a list of paths to its input files, and (b) an output directory. It should rewrite each input file to the output directory with the same name, but deduplicate the content by removing lines that occur more than once in the set of input files. For example, if the input paths are a/1.txt and a/2.txt, and the output directory is b/, your function should write the files b/1.txt and b/2.txt. Implement the adapter [run_exact_line_deduplication] and make sure it passes uv run pytest -k test_exact_line_deduplication.

Implementation: I implemented exact line deduplication by performing two passes over the input files. In the first pass, I computed a hash for each line using mmh3.hash and tracked the global frequency of each line across the entire corpus using a collections.Counter to minimize memory overhead. In the second pass, I reopened each file, evaluated the pre-computed hash for every line, and wrote the line to the corresponding output directory only if its global count was exactly 1.

Code: Code is in cs336-data/cs336_data/implementation.py.

Problem (minhash_deduplication): 8 points

Write a function that takes a list of paths to input files and performs fuzzy document deduplication with minhash and LSH. In particular, your function should compute minhash signatures for each document in the provided list of paths, use LSH with the provided number of bands to identify candidate duplicates, and then compute the true ngram Jaccard similarity between candidate duplicates and remove those that exceed a given threshold. To improve recall (following Penedo et al., 2023), normalize the text before computing minhash signatures and/or comparing Jaccard similarity by lowercasing, removing punctuation, normalizing whitespaces, and removing accents, and applying NFD unicode normalization.

Deliverable: A function that performs fuzzy document deduplication. Your function should take at least the following arguments: (a) a list of paths to its input files, (b) the number of hashes to use for computing minhash signatures, (c) the number of bands to use for LSH, (d) the n-gram length (in words) for computing minhash signatures, and (e) an output directory. You may assume that the number of hashes to use for computing minhash signatures is evenly divisible by the number of bands to use for LSH.

Your function should rewrite each input file to the output directory with the same name, but only writing documents that are either (a) not candidate duplicates and/or (b) are randomly selected to be retained from the clustered buckets. For example, if the input paths are `a/1.txt` and `a/2.txt`, and the output directory is `b/`, your function should write the files `b/1.txt` and `b/2.txt`. Implement the adapter `[run_minhash_deduplication]` and make sure it passes `uv run pytest -k test_minhash_deduplication`.

Implementation: I implemented MinHash deduplication by first normalizing the text of each document by lowercasing, applying NFD Unicode normalization, removing accents, stripping punctuation, and collapsing whitespaces. I then converted the text into sets of word n-grams and generated MinHash signatures using `mmh3` across the specified number of hash seeds. To efficiently find candidate duplicates, I used Locality-Sensitive Hashing (LSH) by dividing the signatures into bands and grouping documents into `defaultdict` buckets. For candidate pairs, I verified their true Jaccard similarity; pairs exceeding the provided threshold were grouped into connected components using a Union-Find data structure. Finally, I randomly selected exactly one document to retain from each cluster and copied its raw contents to the output directory.

Code: Code is in `cs336-data/cs336_data/implementation.py`.